



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Dirección de Transformación Digital



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Estándares de Uso GIT

Dirección Transformación Digital

usm.cl



Introducción



Git es una herramienta de control de versiones distribuida que permite gestionar el código fuente con historial, ramas y colaboración entre múltiples desarrolladores. Esta guía establece cómo usar Git de forma estandarizada y eficiente.

Vamos a ver por qué estandarizar el uso de Git es importante.
Esta guía está pensada para todos los miembros del equipo, desde quienes integran código hasta quienes lo revisan o despliegan.



Objetivo



- Establecer buenas prácticas para el uso de Git.
- Mejorar la colaboración y calidad del código.
- Reducir errores comunes en integración y versionado.
- Facilitar procesos de automatización y CI/CD.

El objetivo es asegurar que todos trabajemos con la misma lógica y estructura, optimizando tiempos y minimizando conflictos

Convenciones de nombres: ramas



Ramas de Desarrollo:

feature/{nombre-descriptivo}

bugfix/{nombre-del-bug}

Ramas de Mantenimiento y Producción

hotfix/{nombre-urgente}

release/{versión}

Las ramas deben seguir una nomenclatura clara y estructurada para facilitar la identificación del propósito de cada una. Utilizamos las siguientes convenciones:

Ramas de Desarrollo

feature/{nombre-descriptivo} → Para nuevas funcionalidades. Ejemplo:
feature/login-oauth

bugfix/{nombre-del-bug} → Para corregir errores en develop. Ejemplo:
bugfix/carga-lenta-dashboard

Ramas de Mantenimiento y Producción

hotfix/{nombre-urgente} → Para correcciones urgentes en main. Ejemplo:
hotfix/error-en-pago

release/{versión} → Para preparar una nueva versión estable antes de pasar a main. Ejemplo: release/v1.2.0



- Utilizar guiones en lugar de espacios
- Usar minúsculas para evitar inconsistencias
- Mantener nombres cortos pero descriptivos
- Para tareas específicas, agregar prefijos opcionales como chore/ para cambios menores

Reglas Generales para Nombres de Ramas

1. Utilizar **guiones** en lugar de espacios (feature/nueva-funcionalidad, y no feature/nueva funcionalidad).
2. Usar **minúsculas** para evitar inconsistencias (bugfix/error-formulario).
3. Mantener nombres **cortos pero descriptivos** (feature/descarga-pdf, y no feature/agregar-funcionalidad-para-descargar-un-pdf).
4. Para tareas específicas, agregar prefijos opcionales como chore/ para cambios menores (chore/actualizar-dependencias).



Convenciones de Nombres: Mensajes Commit

Tipos de commit

- **feat:** Nueva funcionalidad.
- **fix:** Corrección de error.
- **docs:** Cambios en documentación.
- **style:** Cambios de formato, sin afectar código funcional.
- **refactor:** Reestructuración de código sin cambiar su comportamiento.
- **test:** Agregar o modificar pruebas.
- **chore:** Mantenimiento o cambios menores no relacionados con código funcional.
- **perf:** Mejoras de rendimiento.

```
<tipo>(opcional: alcance): <descripción breve en presente>  
[opcional] Descripción más detallada en el cuerpo.  
[opcional] Referencias a issues o tickets.
```

```
feat: agregar autenticación con Google OAuth  
fix: corregir validación en el formulario de login  
refactor(usuario): optimizar carga de datos en perfil
```

Los mensajes de commit deben ser claros, concisos y seguir una estructura estándar para facilitar la comprensión del historial de cambios. Seguimos la convención **Conventional Commits**, que establece un formato estructurado para cada mensaje.

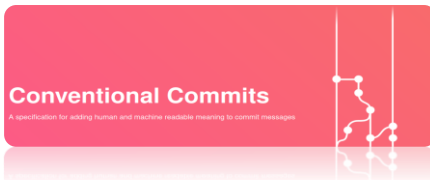
¿Por qué seguir una convención en los commits?

Claridad: Permite entender rápidamente qué cambios se realizaron.

Automatización: Facilita la generación automática de changelogs y versiones.

Historial legible: Mejora la trazabilidad del desarrollo.

Facilita revisiones: Hace que los revisores puedan evaluar los cambios con mayor facilidad.



- Escribir en presente: feat: agrega nueva funcionalidad
- Mantener el mensaje breve y descriptivo.
- Evitar mensajes genéricos como fix: arreglos o update
- Si el cambio es grande, usar el cuerpo del commit para detalles adicionales
- Relacionar commits con issues o tickets, si aplica (fix: corregir error #123).

Reglas generales para los commits

1. Escribir en **presente**: feat: agrega nueva funcionalidad (y no feat: agregó).
2. Mantener el mensaje **breve y descriptivo**.
3. **Evitar mensajes genéricos** como fix: arreglos o update.
4. Si el cambio es grande, usar el cuerpo del commit para detalles adicionales.
5. Relacionar commits con **issues o tickets**, si aplica (fix: corregir error #123).



Estrategia de Versionado

v2.21.2

Seguimos **Versionado Semántico**
MAJOR.MINOR.PATCH

MAJOR: Cambios
incompatibles (ej.
eliminar un método).

MINOR: Nuevas
funcionalidades (sin
romper
compatibilidad).

PATCH:
Correcciones o
mejoras internas
menores.

El versionado es fundamental para mantener un historial estructurado del desarrollo del software. Seguimos el esquema **Versionado Semántico** (MAJOR.MINOR.PATCH) para definir el impacto de cada cambio en el sistema.



Ejemplos de Cambios por Versión

v2.21.2

MAJOR	MINOR	PATCH
Se eliminó o renombró un método público.	Se agregó una nueva funcionalidad	Se corrigió un bug sin alterar interfaces públicas.
Se cambió el formato de una respuesta de API.	Endpoint con variables opcional.	Se optimizó una consulta sin modificar su salida.

Los cambios MAJOR representan modificaciones que son incompatibles con versiones anteriores del software, es decir, al actualizar a esta versión podrían surgir errores o incompatibilidades en el código existente y/o en otras aplicaciones, si no se hacen ajustes adicionales.

Cambios en el formato de respuestas de una API

- **Ejemplo:** Antes: GET /users/{id} devolvía { "name": "Juan", "age": 30 }, ahora devuelve { "fullName": "Juan Pérez", "yearsOld": 30 }.
- **Problema:** Clientes que consumían la API esperando name y age ya no recibirán esos campos.

Los cambios MINOR son aquellos que introducen nuevas funcionalidades sin afectar la compatibilidad con versiones anteriores. Esto significa que los desarrolladores que usaban versiones anteriores pueden actualizar sin necesidad de modificar su código.

Agregar una nueva API o endpoint sin afectar los existentes

- **Ejemplo:** Se añade GET /users/profile en una API sin modificar GET /users/{id}.
- **Impacto:** Se agrega funcionalidad sin romper compatibilidad con versiones anteriores.

Los cambios PATCH son aquellos que **corrigen errores o hacen mejoras menores sin afectar la compatibilidad** con versiones anteriores. Esto significa que los desarrolladores pueden actualizar a la nueva versión sin realizar ajustes en su código.

Arreglo de una vulnerabilidad de seguridad sin modificar la API

- **Ejemplo:** Se corrige una vulnerabilidad de XSS en la sanitización de entradas sin modificar los endpoints de la API.
- **Impacto:** La seguridad mejora sin afectar la compatibilidad con versiones anteriores.

Aquí vemos mas ejemplos prácticos. Recordemos: cada número transmite una expectativa al resto del equipo y usuarios del sistema.



Etiquetado de versiones en Git

```
1 git tag -a v1.2.0 -m "Lanzamiento de la versión 1.2."  
2 git push origin v1.2.0
```

Cada versión estable debe ser etiquetada en Git para su fácil identificación y recuperación

En Git, los **tags** (etiquetas) marcan **versiones importantes** del proyecto, como lanzamientos.

Sirven para:

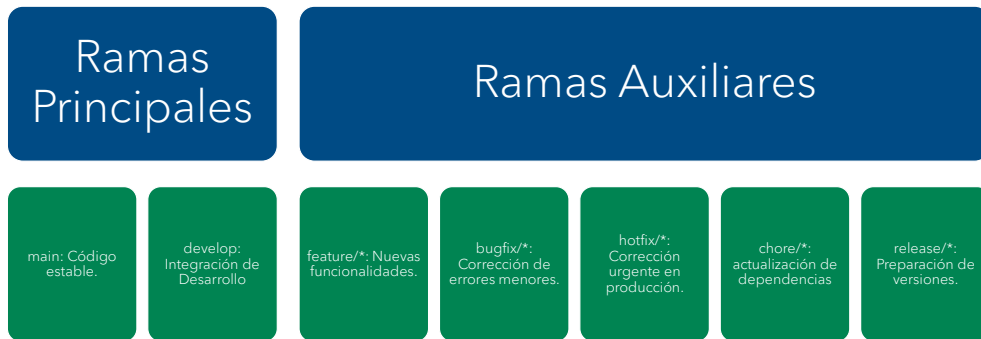
- Identificar versiones (ej: v1.0, v2.1.3).
- Volver fácilmente a un punto específico.

```
git tag v1.0      # crea un tag  
git tag          # lista los tags  
git checkout v1.0 # ir a ese punto
```

Los tags **no cambian**, son como "fotos" del estado del repo en un momento clave.



Flujo de Trabajo en Git: Desarrollo



Git Flow nos permite separar ambientes y tipos de trabajo. Cada tipo de rama tiene un propósito claro que veremos en acción.

Ramas Principales

main: Contiene el código estable y listo para producción. Solo se realizan merges desde release o hotfix.

develop: Es la rama base para el desarrollo continuo. Todas las nuevas funcionalidades y correcciones de errores se integran aquí antes de pasar a producción.

Ramas Auxiliares

feature/{nombre}: Se crean a partir de develop para desarrollar nuevas funcionalidades sin afectar la estabilidad del código base. Una vez completadas, se fusionan en develop.

bugfix/{nombre}: Se usan para corregir errores identificados en develop. Una vez solucionados, se integran nuevamente en develop.

chore/{nombre}: Se usa para cambios o correcciones que tengan que ver con archivos de configuración o dependencias que no necesariamente es una funcionalidad

hotfix/{nombre}: Para correcciones críticas en main. Se crean a partir de main y, una vez corregido el problema, se fusionan tanto en main como en develop para mantener la consistencia.

release/{versión}: Se crean a partir de develop cuando se prepara una nueva



versión. En esta rama solo se realizan ajustes menores y correcciones. Luego, se fusiona en main y develop, asegurando que los cambios se propaguen correctamente.



Flujo de Trabajo en Git: Trabajo con scripts, reportes y consulta

Ramas Principales

main: versión estable del repositorio

Ramas Auxiliares

work/nombre-usuario: : trabajo individual

review/nombre-feature: si alguien va a revisar antes de fusionar

Aunque Git es ampliamente conocido por su uso en proyectos de desarrollo de software, también es una herramienta poderosa y altamente recomendable para quienes trabajan con **consultas SQL, scripts de análisis de datos, reportes en hojas de cálculo o notebooks interactivos (como Jupyter)**.

Establecer un estándar de uso de Git para este tipo de trabajos ayuda a mejorar la organización, trazabilidad, colaboración y control de cambios, incluso si no se trata de una aplicación de software en sí misma.

Definir una estructura y buenas prácticas para utilizar Git como sistema de control de versiones en el trabajo con archivos individuales o por lotes, tales como:

- Scripts SQL (.sql)
- Notebooks Jupyter (.ipynb)
- Reportes y resultados (.csv, .xlsx, .pdf, etc.)



Estructura recomendada del repositorio

```
/
├─ consultas/      # Scripts SQL
├─ reportes/       # Archivos de resultados (.csv )
├─ notebooks/     # Notebooks Jupyter (.ipynb)
├─ .gitignore
└─ README.md
```

- **Evita archivos grandes en Git**

- No subir .csv, .xlsx o .parquet muy pesados.
- Usa enlaces a carpetas compartidas o datos anonimizados para cumplir con leyes de privacidad. Es importante no subir información sensible real.

README: El archivo README.md debe describir:

- Qué contiene cada carpeta.
- Qué hace cada script importante.
- Cómo ejecutar o reutilizar consultas si aplica.
- Aclaraciones sobre nombres, fechas, filtros, etc.

Proceso de Integración

Integración de feature y bugfix en develop:



El proceso de integración define los pasos necesarios para agregar cambios al código base de manera segura y ordenada. Para cada nueva funcionalidad o corrección de errores, se sigue el siguiente procedimiento:

Crear una rama: A partir de develop, se genera una nueva rama con la nomenclatura adecuada según el tipo de cambio.

Realizar cambios y commits: Se desarrollan las modificaciones en la nueva rama, asegurando que los commits sean descriptivos y sigan las convenciones establecidas.

Sincronizar con develop: Antes de realizar el merge, es recomendable actualizar la rama con los últimos cambios de develop mediante `git pull --rebase develop`.

Crear un Merge Request (MR): Se genera un MR hacia develop, proporcionando una descripción clara de los cambios realizados.

Revisión y aprobación: Un miembro del equipo revisa el código, sugiriendo cambios si es necesario. El MR no debe fusionarse sin una revisión adecuada.

Merge y eliminación de la rama: Una vez aprobado, se realiza el merge de la rama en develop. Si la rama ya no es necesaria, se elimina para mantener el repositorio limpio.



Proceso de Integración

Integración de release en main:



Quando el código en develop está listo para una nueva versión, se crea una rama release/{versión}.

Se realizan pruebas y ajustes menores en esta rama.

Una vez validada, la rama release se fusiona en main.

Se etiqueta la nueva versión en main (git tag vX.Y.Z).

Finalmente, la rama release también se fusiona en develop para asegurar que los cambios se mantengan en desarrollo.



Proceso de Integración

Integración de hotfix en main y develop:



Si se detecta un error crítico en producción, se crea una rama hotfix/{nombre} desde main.

Se realiza la corrección y se confirma con un commit claro (fix: corregir error crítico en autenticación).

Se genera un Merge Request hacia main para su revisión y aprobación.

Una vez aprobado, se fusiona en main y se etiqueta con la versión corregida (git tag vX.Y.Z+1).

Luego, el mismo hotfix se fusiona en develop para evitar regresiones en futuras versiones.



Proceso de Integración

Fusión de Múltiples Features antes de develop:

Crear una rama de integración. Ej. feature/integracion-featureX-featureY

Fusionar las ramas feature en la rama intermedia

Probar y validar cambios

Crear un Merge Request a develop

Eliminar ramas innecesarias

Si se usa esta estrategia, debe ser una excepción y no la norma. Cada equipo debe evaluar si realmente aporta valor antes de implementarla.

En algunos casos, cuando varias features están relacionadas o afectan las mismas áreas del código, los desarrolladores pueden optar por fusionarlas en una rama intermedia antes de integrarlas en develop. Sin embargo, **esta práctica solo se recomienda en casos específicos**

¿Cuándo usar esta estrategia?

Cuando varias features dependen entre sí y necesitan pruebas conjuntas antes de integrarse en develop.

Cuando hay un equipo trabajando en una gran funcionalidad y necesitan consolidar su código.

Cuando se espera que las ramas feature/* generen conflictos entre sí, y se prefieren resolver antes de develop.

¿Cuándo evitar esta estrategia?

Cuando cada feature puede integrarse individualmente en develop.

Si la integración en una rama intermedia se vuelve un cuello de botella y ralentiza el proceso.

Si oculta problemas que deberían resolverse directamente en develop

Crear una rama de integración

Fusionar las ramas feature en la rama intermedia

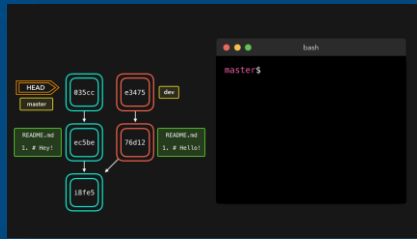
```
git checkout feature/integracion-featureX-featureY
git merge feature/funcionalidadX
git merge feature/funcionalidadY
```

Probar y validar cambios

Se ejecutan pruebas y revisiones antes de fusionar en develop.



Con este enfoque, garantizamos que todas las ramas se integren adecuadamente según su propósito, evitando errores y asegurando un desarrollo estructurado



Manejo de Conflictos

Pasos para resolver conflictos:

Identificar el conflicto, ver con git status.

Editar el archivo y resolver el conflicto.

Marcar como resuelto con git add.

Confirmar con commit: fix(merge): resolver conflicto en archivo X

También puede usarse Visual Studio Code para resolver visualmente.

El manejo de conflictos en Git es una parte esencial del trabajo colaborativo, ya que pueden ocurrir cuando múltiples desarrolladores modifican la misma parte del código en diferentes ramas.

¿Por qué ocurren los conflictos en Git?

Edición concurrente de archivos: Dos o más personas modifican la misma línea en el mismo archivo.

Eliminación y modificación simultánea: Un desarrollador elimina un archivo mientras otro lo edita.

Rebases o merges con cambios incompatibles: Se intenta fusionar dos ramas con modificaciones en los mismos archivos.

Paso 1: Identificar el conflicto

Cuando Git detecta un conflicto, mostrará un mensaje indicando qué archivos tienen problemas. git status

Los archivos en conflicto aparecerán marcados como both modified.

Paso 2: Editar el archivo y resolver el conflicto.

Git marcará las secciones en conflicto con un HEAD y unas flechas

Edita el archivo para seleccionar los cambios correctos o combinar ambas versiones según sea necesario.

Elimina los marcadores <<<<<<, ===== y >>>>>>.

Paso 3: Marcar el conflicto como resuelto

Después de editar el archivo y resolver los conflictos manualmente, marca el archivo como resuelto utilizando:

Paso 4: Finalizar el merge o rebase

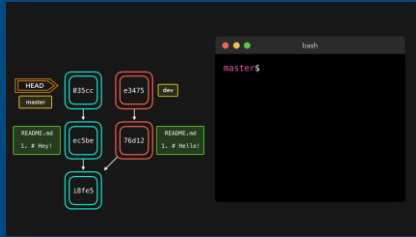
Si estabas haciendo un merge, sigue la convención de commits al confirmar la resolución



del conflicto:

git rebase --continue

Buenas prácticas para evitar conflictos



Revisar los Merge Requests antes de fusionar

Con estas prácticas, el manejo de conflictos en Git se vuelve más eficiente y reduce la posibilidad de problemas en el código compartido.



Merge Requests en GitLab

Proceso para integrar cambios:

Crear Merge Request en GitLab.

Agregar descripción clara.

Asignar revisores.

Esperar aprobación y luego fusionar.

Un **Merge Request (MR)** es una solicitud para fusionar cambios de una rama en otra dentro del flujo de trabajo del repositorio.

¿Por qué usar Merge Requests?

Revisión de código: Permite que otros miembros del equipo revisen los cambios antes de integrarlos en la rama principal.

Prevención de errores: Reduce la posibilidad de introducir fallos en el código base.

Historial claro de cambios: Documenta qué modificaciones se han realizado y por qué.

Facilita la colaboración: Permite discusiones y sugerencias antes de aprobar cambios

Agregar una Descripción Clara

Un buen MR debe incluir:

Título conciso (feat: agregar autenticación con Google OAuth).

Descripción detallada con el propósito del cambio, impacto y si hay dependencias.

Referencias a issues o tickets (Closes #123 si soluciona un problema específico).

Revisar y Aprobar

Se asigna un revisor que evalúa el código y deja comentarios.

Se pueden realizar cambios si el revisor sugiere mejoras.

Se aprueba y se fusiona cuando todo está validado.

Fusionar y Eliminar la Rama

Si el MR es aprobado, se realiza el merge desde GitLab :



Luego, se elimina la rama para mantener el repositorio limpio:



Merge Requests en GitLab

Buenas Prácticas:

Crear MRs pequeños y enfocados: Facilita la revisión y reduce conflictos.

Usar nombres claros en las ramas y commits.

Hay que asegurar que los cambios pasen las pruebas automatizadas antes del MR.

Incluir capturas de pantalla o ejemplos si es necesario.

Revisar MRs de otros compañeros para fomentar la colaboración.



Uso del archivo .gitignore

Cómo usar .gitignore :

Crear un archivo .gitignore en la raíz del repositorio si no existe. → touch .gitignore

Agregar reglas de exclusión dentro de .gitignore.

Aplica el .gitignore a archivos aún no versionados

```
1 git add .
2 git commit -m "chore(gitignore): actualizar reglas de exclusión"
3
```

[Lista oficial de .gitignore en GitHub.](#)

El archivo .gitignore se utiliza para **excluir archivos y carpetas** del control de versiones en Git, evitando que archivos innecesarios o sensibles sean agregados al repositorio.

¿Por qué usar .gitignore?

Evita subir archivos temporales, logs o credenciales sensibles.

Mantiene el repositorio limpio y organizado.

Reduce el tamaño del repositorio y evita conflictos innecesarios.

Un .gitignore bien configurado mantiene el repositorio limpio y seguro. Es importante no subir configuraciones locales o secretos.



Automatización con Hooks y CI/CD

Tipos comunes de hooks:

- **Pre-commit:** Se ejecuta antes de realizar un commit, ideal para validar estilo del código o ejecutar pruebas unitarias.
- **Commit-msg:** Permite validar el formato de los mensajes de commit según la convención establecida.
- **Pre-push:** Ejecuta pruebas automáticas antes de permitir enviar cambios al repositorio remoto.

Herramientas recomendadas para automatización

- Husky (JavaScript)
- Pre-commit (Python y lenguajes en general)
- GitLab CI/CD (Automatización desde GitLab)

La hooks en Git son scripts automáticos que se ejecutan en momentos específicos del ciclo de vida de Git, permitiendo automatizar tareas repetitivas o validar condiciones antes o después de ciertas acciones (por ejemplo, antes de hacer un commit, o al subir cambios al repositorio).

¿Por qué usar hooks en Git?

Mejor calidad del código: Permiten validar el código automáticamente antes de subir cambios al repositorio.

Prevención de errores: Evitan que código defectuoso o que no cumpla con estándares se fusione al código base.

Automatización: Facilitan el trabajo de desarrollo, ejecutando automáticamente tareas repetitivas o esenciales.

Tipos comunes de hooks:

Pre-commit: Se ejecuta antes de realizar un commit, ideal para validar estilo del código o ejecutar pruebas unitarias.

Commit-msg: Permite validar el formato de los mensajes de commit según la convención establecida.

Pre-push: Ejecuta pruebas automáticas antes de permitir enviar cambios al repositorio remoto.

Herramientas recomendadas para automatización:

Husky (JavaScript):

Herramienta para ejecutar scripts automáticos antes o después de commits, push, etc.

Pre-commit (Python y lenguajes en general): Ejecuta tareas antes de realizar un commit.

GitLab CI/CD (Automatización desde GitLab): Para automatizar pruebas, análisis de código,



generación de builds y despliegues



Comparación entre ecosistemas

Propósito	JavaScript / TypeScript	Python
Formatear código	Prettier	Black
Ordenar imports	(Incluido en Prettier o import-sort)	isort
Linting (estilo / errores)	ESLint	flake8
Automatizar hooks Git	Husky + lint-staged	pre-commit
Validar mensajes de commit	commitlint + husky	(se puede usar también con husky) O crear script manual



Buenas prácticas al usar hooks y automatización

Mantener scripts simples y rápidos para no ralentizar el desarrollo.

Documentar claramente qué hace cada hook y cómo resolver problemas si fallan.

Comunicar claramente al equipo qué automatizaciones están habilitadas para evitar sorpresas.



Reglas Adicionales

Buenas Prácticas:

- Evitar commits grandes
- No hacer force push en main o develop
- Mantener el repositorio actualizado
- No trabajar directamente en main o develop
- Eliminar ramas que ya no sean necesarias.
- Usar etiquetas para versiones estables
- Asegurar revisiones antes del merge

Reglas Importantes para el Uso de Git

Evitar commits grandes: Es preferible hacer cambios incrementales y bien documentados.

No hacer force push en main o develop para evitar sobrescribir cambios de otros desarrolladores.

Mantener el repositorio actualizado: Usar `git pull --rebase` antes de comenzar a trabajar para evitar conflictos innecesarios.

No trabajar directamente en main o develop, siempre usar ramas `feature`, `bugfix` o `hotfix`.

Eliminar ramas que ya no sean necesarias: Después de fusionar una rama, eliminarla con `git branch -d nombre-rama` y `git push origin --delete nombre-rama`.

Usar etiquetas para versiones estables: Versionar lanzamientos con `git tag -a vX.Y.Z -m "Descripción de la versión"`.

Asegurar revisiones antes del merge: No fusionar cambios sin una revisión de otro desarrollador.

Documentar los cambios: Cada commit y merge request debe incluir una descripción clara de qué y por qué se hicieron los cambios.

Evitar código temporal o de prueba en los commits: Siempre revisar que no se suban archivos innecesarios o cambios temporales.

Configurar correctamente .gitignore para evitar incluir archivos de configuración local, dependencias innecesarias o archivos temporales.

Siguiendo estas reglas, garantizamos un flujo de trabajo eficiente y un código bien mantenido.

Estas reglas complementan todo lo anterior. Nos ayudan a trabajar en equipo sin



pisarnos los cambios y manteniendo la historia clara.



- Estándares claros permiten una colaboración efectiva.
- Git es más poderoso cuando se usa de forma estructurada.
- La automatización y la disciplina técnica hacen más eficiente el trabajo en equipo.

Conclusión

Finalizamos recordando que un buen uso de Git no es solo técnico, sino también organizacional. Es parte de cómo trabajamos como equipo.



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Gracias

Alejandra Reinoso Ríos

Arquitecta de Software y Estándares de Desarrollo

Dirección Transformación Digital

usm.cl